

TECNOLÓGICO DE COSTA RICA

Escuela de Ingeniería en Computadores
Fundamentos de Sistemas Computacionales

Documento de Atributos de Acreditación

Proyecto 2 — Radar 2D

Integrantes:

Cristopher Viquez Quiros, 2026014559
David Ramirez Alvarez, 2026008216

Profesor Leonardo Araya
I Semestre 2026

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción	3
2. Identificación de Problemas.....	3
2.1 Problema 1: Estimación de trayectoria a partir de lecturas ruidosas y escasas	3
2.2 Problema 2: Limitación de velocidad de barrido por el tiempo de respuesta del sensor ultrasónico	4
2.3 Problema 3: Sincronización de bases de tiempo independientes entre Arduino y la computadora	4
3. Planteamiento de la Solución	5
3.1 Solución al Problema 1 – Estimación de trayectoria	5
Alternativas evaluadas.....	5
3.2 Solución al Problema 2 – Velocidad de barrido vs. resolución angular	6
Alternativas evaluadas.....	6
3.3 Solución al Problema 3 – Sincronización de tiempo entre Arduino y Python.....	7
Alternativas evaluadas.....	7
4. Conclusiones	10

1. Introducción

El presente documento forma parte de los entregables del Proyecto 2 — Radar 2D, desarrollado para el curso Fundamentos de Sistemas Computacionales del Tecnológico de Costa Rica. Su propósito es evidenciar la aplicación del atributo de acreditación Conocimiento de Ingeniería (CI), entendido como la capacidad de aplicar conocimientos de matemáticas, ciencias naturales, fundamentos de la ingeniería y conocimientos especializados para la solución de problemas complejos de ingeniería.

El proyecto consistió en el diseño y construcción de un radar 2D funcional, compuesto por un sensor ultrasónico de proximidad (HC-SR04) montado sobre un servomotor controlado por un Arduino, capaz de realizar un barrido angular de 180° e identificar objetos dentro de su campo de detección. El sistema incluye además un LED rojo de alerta que parpadea de forma continua e independiente del ciclo de barrido, replicando la señalización utilizada en estructuras altas para advertir a aeronaves. Los datos capturados se transmiten por el puerto serial hacia una interfaz gráfica desarrollada en Python (Pygame), la cual visualiza en tiempo real la posición de los objetos, su velocidad estimada y una proyección de su trayectoria futura basada en el modelo de movimiento parabólico.

Durante el desarrollo del proyecto, el equipo de trabajo enfrentó diversos problemas técnicos de naturaleza distinta — uno asociado al procesamiento de señales y estimación de trayectorias, otro al diseño electrónico del sensor, y un tercero a la sincronización de sistemas distribuidos en tiempo real (Arduino—computadora). A continuación se documentan estos tres problemas, las alternativas de solución consideradas para cada uno, y la justificación técnica de la opción finalmente implementada.

2. Identificación de Problemas

Se identificaron tres problemas técnicos complejos durante el desarrollo del proyecto, descritos a continuación junto con su causa raíz y el impacto que tuvieron sobre el funcionamiento del sistema.

2.1 Problema 1: Estimación de trayectoria a partir de lecturas ruidosas y escasas

El radar debe predecir la posición futura de un objeto a partir de su historial de lecturas, pero dicho historial está limitado a muestras tomadas en instantes discretos del barrido (cada vez que el ángulo del servo coincide con la posición del objeto). Esto implica que, para un objeto en movimiento, se cuenta típicamente con solo 2 o 3 lecturas recientes antes de necesitar una predicción, y dichas lecturas están sujetas al ruido propio del sensor ultrasónico (variaciones de $\pm 1\text{-}2$ cm por reflexión irregular del eco). Adicionalmente, las lecturas llegan en coordenadas polares (ángulo, distancia), mientras que el modelo de movimiento parabólico requiere coordenadas cartesianas y una estimación de velocidad vectorial (v_x , v_y), lo cual añade una capa de transformación matemática antes de poder aplicar las ecuaciones de movimiento.

El impacto de este problema es directo sobre el cumplimiento del objetivo específico 3 del proyecto: si la velocidad estimada es inestable debido al ruido, la trayectoria predicha oscila erráticamente entre lecturas sucesivas, restando utilidad y credibilidad a la funcionalidad de predicción mostrada en la interfaz gráfica.

2.2 Problema 2: Limitación de velocidad de barrido por el tiempo de respuesta del sensor ultrasónico

El sensor HC-SR04 mide distancia emitiendo un pulso ultrasónico y midiendo el tiempo que tarda el eco en regresar (función `pulseIn`). Esta medición es inherentemente bloqueante: el microcontrolador debe esperar activamente hasta que el pulso de eco se complete o hasta que expire un tiempo límite (timeout). Con un rango máximo de 100 cm, el tiempo de vuelo del sonido y el timeout de seguridad necesario para evitar bloqueos indefinidos (en caso de que no exista un objeto que refleje el pulso) imponen un límite inferior al tiempo que toma cada medición individual.

Esto genera un conflicto de diseño directo: a mayor resolución angular (pasos más pequeños entre una medición y la siguiente), mayor es el tiempo total que toma completar un barrido de 180°, reduciendo la tasa de refresco efectiva del radar. Por el contrario, aumentar la velocidad del barrido (pasos más grandes o menor espera entre mediciones) reduce la resolución angular y aumenta el riesgo de que objetos pequeños o en movimiento rápido no sean detectados en absoluto durante un ciclo de barrido.

2.3 Problema 3: Sincronización de bases de tiempo independientes entre Arduino y la computadora

El cálculo de velocidad de los objetos ($\text{velocidad} = \text{distancia} / \text{tiempo}$) requiere conocer con precisión el intervalo de tiempo transcurrido entre dos lecturas consecutivas del mismo objeto. Sin embargo, el Arduino y la computadora que ejecuta la interfaz gráfica en Python son dos sistemas independientes, cada uno con su propio reloj interno: el Arduino mide tiempo relativo desde su encendido (`millis()`), mientras que Python mide tiempo mediante el reloj del sistema operativo (`time.time()`).

Si el cálculo de velocidad se realizara utilizando una marca de tiempo generada en el Arduino y transmitida por serial, sería necesario sincronizar ambos relojes y compensar la latencia de transmisión y lectura del puerto serial (la cual no es constante, pues depende del sistema operativo, del buffer del puerto y de la carga del hilo de lectura en Python). Una sincronización deficiente introduce un error sistemático en el cálculo de velocidad que se traduce directamente en una mala estimación de la trayectoria parabólica predicha.

3. Planteamiento de la Solución

Para cada uno de los tres problemas identificados se evaluaron al menos tres alternativas de solución, comparándolas según criterios puramente técnicos: precisión, complejidad de implementación, costo computacional y robustez ante las condiciones reales del hardware utilizado. A continuación se presenta el análisis para cada problema y la justificación de la opción seleccionada.

3.1 Solución al Problema 1 — Estimación de trayectoria

Alternativas evaluadas

Opción	Descripción	Ventaja principal	Desventaja principal
Filtro de Kalman	Filtro recursivo óptimo que estima el estado (posición, velocidad) de un sistema dinámico a partir de mediciones ruidosas, ponderando la incertidumbre del modelo y del sensor.	Estimación estadísticamente óptima; suaviza el ruido de forma robusta y converge incluso con mediciones imprecisas.	Requiere modelar matrices de covarianza y de transición de estado, además de sintonizar parámetros de ruido del proceso y del sensor. Complejidad de implementación alta para el alcance del curso.
Promedio móvil simple	Calcular la velocidad como el promedio de las últimas N lecturas de distancia/tiempo, suavizando el ruido por promediado.	Muy simple de implementar y computacionalmente económico.	Introduce retraso (lag) en la estimación: reacciona lento a cambios reales de velocidad, lo cual es inaceptable si el objeto acelera o cambia de dirección.
Movimiento parabólico con velocidad por diferencias finitas (2 últimos puntos)	Se asume trayectoria parabólica simple (como sugiere el enunciado del proyecto). La velocidad vectorial (v_x , v_y) se calcula por diferencia finita entre las dos lecturas cartesianas más recientes, y se proyecta la posición futura	Bajo costo computacional (apto para ejecutarse en tiempo real a 30+ fps); no requiere sintonizar parámetros estadísticos; coherente con el alcance	Más sensible al ruido instantáneo que un Kalman, al depender de solo dos muestras.

Opción	Descripción	Ventaja principal	Desventaja principal
		pedagógico definido en el enunciado del proyecto, que explícitamente sugiere el modelo parabólico simple.	

Se escogió el modelo de movimiento parabólico por diferencias finitas porque es una solución sencilla y suficiente para los objetivos del proyecto. Aunque el Filtro de Kalman puede manejar mejor el ruido en las mediciones, implementarlo correctamente requiere trabajar con matrices y ajustar varios parámetros, lo que está fuera del alcance del proyecto. Además, el enunciado sugiere utilizar un modelo de trayectoria parabólica simple.

Por otro lado, el promedio móvil se descartó porque genera un pequeño retraso al calcular la velocidad, lo que afecta la precisión de la predicción en tiempo real. Para disminuir el impacto del ruido en el modelo elegido, se agregó un sistema de agrupación angular mediante buckets de 8° , que permite asociar lecturas consecutivas al mismo objeto y reduce la posibilidad de combinar datos de objetos diferentes al calcular su velocidad.

3.2 Solución al Problema 2 — Velocidad de barrido vs. resolución angular

Alternativas evaluadas

Opción	Descripción	Ventaja principal	Desventaja principal
Medición por interrupciones (timer hardware)	Configurar un timer del microcontrolador para medir el ancho del pulso de eco mediante interrupciones de hardware, en vez de esperar de forma bloqueante con <code>pulseIn()</code> .	Libera al procesador para realizar otras tareas (mover el servo, parpadear el LED) mientras espera el eco, eliminando el bloqueo.	Requiere manejo directo de registros del microcontrolador (Timer/Counter, ICP), lo cual añade complejidad y reduce la portabilidad del código entre distintos modelos de Arduino.
Reducir el timeout de <code>pulseIn</code> al mínimo teórico	Calcular el timeout exacto necesario para el rango máximo de 100 cm (≈ 5.8 ms de ida y vuelta) y usar ese valor en vez de un timeout genérico más largo.	Reduce el tiempo de espera en mediciones sin eco (objeto fuera de rango), acelerando el barrido en esos casos.	No resuelve el problema de fondo: cada medición sigue siendo bloqueante mientras dura, y el timeout ajustado no

Opción	Descripción	Ventaja principal	Desventaja principal
			mejora el caso en que sí hay eco real.
Balancear el paso angular y el delay de muestreo de forma empírica	Fijar un paso de barrido de 1-2° con un timeout de pulselsn de 30 ms (suficiente margen de seguridad) y delay de movimiento del servo de 15-30 ms, validando experimentalmente la tasa de refresco resultante contra la velocidad real de los objetos a detectar.	Solución simple, sin dependencias de hardware específico, fácil de ajustar y depurar; mantiene el código portable y legible.	Es una solución de compromiso (trade-off), no elimina el límite físico del sensor; la tasa de refresco máxima sigue acotada por la física del sonido.

Se eligió un ajuste empírico de los parámetros del paso angular y la temporización porque fue la opción más práctica para el desarrollo del proyecto. Aunque medir por interrupciones de hardware es una solución más completa, implica trabajar con registros internos del microcontrolador (Timer/Counter), algo que va más allá de lo visto en el curso y que además haría el código menos compatible con diferentes placas Arduino.

También se consideró reducir el tiempo de espera al mínimo posible, pero esa alternativa solo mejora una parte del problema y no elimina el bloqueo que ocurre durante la medición. En cambio, ajustar los parámetros mediante pruebas directas con el hardware permitió encontrar un buen equilibrio entre la velocidad de actualización de la interfaz (alrededor de 30 veces por segundo) y una resolución angular adecuada para el funcionamiento del sistema.

3.3 Solución al Problema 3 — Sincronización de tiempo entre Arduino y Python

Alternativas evaluadas

Opción	Descripción	Ventaja principal	Desventaja principal
Timestamp generado en Arduino y transmitido por serial	El Arduino añade su valor de millis() a cada lectura enviada, y Python calcula el delta de tiempo usando esos timestamps.	El tiempo de origen de la medición es exacto en el dominio del Arduino.	millis() del Arduino y time.time() de Python no son comparables directamente (distintas épocas/orígenes); requeriría un protocolo de sincronización inicial (handshake) y resincronización periódica para corregir el drift

Opción	Descripción	Ventaja principal	Desventaja principal
			entre ambos relojes, añadiendo complejidad al protocolo serial.
Protocolo de sincronización con sello de tiempo NTP-like	Implementar un intercambio de mensajes al inicio de la conexión para estimar el offset y la latencia del enlace serial, similar al protocolo NTP, y aplicar esa corrección a cada timestamp recibido.	Es la solución más precisa en teoría, usada en sistemas distribuidos reales.	Sobre-ingeniería para el caso de uso: la latencia del puerto serial USB es del orden de milisegundos, mientras que el sistema bajo prueba opera con tasas de cambio de varios cm/s, donde un error de pocos milisegundos no es significativo. Añade complejidad de protocolo no justificada por la precisión que realmente se necesita.
Timestamp único generado en Python al momento de la recepción	Ignorar el tiempo del Arduino y registrar time.time() en Python en el instante exacto en que cada línea es leída del puerto serial, usando ese valor como base de tiempo única y consistente para todos los cálculos de velocidad.	Una sola fuente de verdad temporal (un solo reloj), eliminando por completo el problema de sincronización entre relojes distintos; implementación trivial; la latencia de transmisión serial ($\approx 1\text{-}2$ ms a 9600 baudios) es despreciable frente a la escala de tiempo del fenómeno medido (movimiento de objetos a velocidades de cm/s).	El timestamp no corresponde exactamente al instante físico en que el sensor tomó la medición, sino al instante en que Python la recibió, introduciendo un pequeño y constante offset de latencia.

Se decidió utilizar una única fuente de tiempo (`time.time()` en Python) para calcular la velocidad de los objetos, ya que es una solución simple y suficiente para las necesidades del proyecto. Otras opciones, como usar el tiempo del Arduino o implementar un sistema de sincronización entre ambos dispositivos, agregaban una complejidad innecesaria sin ofrecer una mejora significativa en los resultados.

Además, la comunicación por USB tiene un retraso muy pequeño en comparación con la velocidad a la que se mueven los objetos que se detectan, por lo que ese tiempo no afecta de forma importante los cálculos. Al utilizar un solo reloj, el de la computadora, se evita tener que sincronizar dos referencias de tiempo distintas, haciendo el código más sencillo y manteniendo la precisión necesaria para calcular la velocidad y predecir la trayectoria.

4. Conclusiones

- Para resolver los problemas del proyecto fue necesario aplicar conceptos de cinemática, como el movimiento parabólico, además de ideas básicas de filtrado de ruido y sistemas en tiempo real. Esto permitió combinar conocimientos de distintas áreas y no solo la programación del microcontrolador.
- En los tres casos se eligieron soluciones que fueran prácticas y adecuadas para el proyecto. Aunque existían opciones más avanzadas, como el Filtro de Kalman, las interrupciones de hardware o un protocolo de sincronización, estas añadían más complejidad de la necesaria y no representaban una mejora importante para los objetivos del curso.
- Analizar y comparar varias alternativas antes de implementar una solución ayudó a tomar decisiones mejor fundamentadas. Además, permitió entender las ventajas y desventajas de cada opción y desarrollar un sistema que cumple con los requisitos del proyecto de una manera sencilla y eficiente.